04 — JOCKY Language Reference

Philosophy

JOCKY's syntax is deliberately unlike any mainstream language. Every keyword is an ordinary English word that has no special meaning to existing AV parsers. The grammar is simple enough to implement in a weekend but expressive enough to write real forensic tools.

File Structure

A `.jk` file contains one or more **function definitions**. Nothing else is allowed at the top level. No import statements, no global variables, no class definitions.

```
func name(params) -> return_type {
    statements
}

func another_function() -> nothing {
    ...
}
```

Entry point: The compiler looks for a function named exactly `start`. This is the function called when the program runs. It must have the signature `func start() -> nothing`.

Formal Grammar (BNF)

```
<program>       ::= <function_def>*

<function_def>  ::= "func" IDENTIFIER "(" <param_list>? ")" "->" <type> <block>

<param_list>    ::= <param> ("," <param>)*
<param>         ::= IDENTIFIER ":" <type>

<type>          ::= "num" | "dec" | "text" | "flag" | "raw" | "nothing"

<block>         ::= "{" <statement>* "}"

<statement>     ::= <var_decl>
                  | <assignment>
                  | <if_stmt>
                  | <loop_stmt>
                  | <return_stmt>
                  | "stop"
                  | "skip"
                  | <expression>
```

```
<var_decl>      ::= "var" IDENTIFIER ":" <type> ":=" <expression>
<assignment>    ::= IDENTIFIER "<-" <expression>
<if_stmt>       ::= "check" "(" <expression> ")" <block> ("otherwise" <block>)?
<loop_stmt>     ::= "loop" "(" <expression> ")" <block>
<return_stmt>   ::= "give" <expression>?

<expression>    ::= <logic>
<logic>         ::= <comparison> (("also" | "or") <comparison>)*
<comparison>    ::= <arithmetic> (("is"|"isnt"|"gt"|"lt"|"gte"|"lte")
<arithmetic>)?
<arithmetic>    ::= <term> (("+" | "-") <term>)*
<term>          ::= <unary> (("*" | "/" | "mod") <unary>)*
<unary>         ::= "flip" <unary> | "-" <unary> | <primary>
<primary>       ::= NUMBER
                  | FLOAT
                  | STRING
                  | "yes" | "no" | "empty"
                  | IDENTIFIER "(" <arg_list>? ")"
                  | IDENTIFIER
                  | "(" <expression> ")"
<arg_list>      ::= <expression> ("," <expression>)*
```

Tokens

Comments

```
## Everything after ## on the same line is a comment
var x : num := 5  ## inline comment
```

Identifiers

```
func_name   my_variable   x   count   found_it
```

Start with a letter or _, followed by letters, digits, or _. Case-sensitive. Keywords cannot be identifiers.

Numbers

```
42        0        1000000
```

One or more decimal digits. Parsed as 64-bit signed integer.

Floats

```
3.14      0.5      -1.0      100.0
```

Digits, then `.`, then more digits. Both parts required.

Strings

```
`hello world`
`svchost.exe`
`HKEY_LOCAL_MACHINE\SOFTWARE`
```

Enclosed in backticks. Cannot span multiple lines. The content is the raw text between the backticks — no escape sequences.

Operators

```
:=    assignment (declaration only)
<-    assignment (update only)
->    return type separator
+  -  *  /  mod    arithmetic
is  isnt  gt  lt  gte  lte    comparison
also  or  flip    boolean logic
```

Types

`num` — 64-bit signed integer

Range: −9,223,372,036,854,775,808 to 9,223,372,036,854,775,807.

```
var count : num := 0
var pid   : num := 1234
var n     : num := proc_count(procs)
```

LLVM type: `i64`. C equivalent: `int64_t`.

Operations: `+`, `-`, `*`, `/` (integer division), `mod`, `is`, `isnt`, `gt`, `lt`, `gte`, `lte`, unary `-`.

`dec` — 64-bit floating-point

IEEE 754 double-precision. ~15–17 significant decimal digits.

```
var ratio  : dec := 3.14
var half   : dec := 1.0 / 2.0
var result : dec := 5 + 2.5    ## auto-converts 5 to 5.0
```

LLVM type: `double`. C equivalent: `double`.

Mixing `num` and `dec` in arithmetic produces `dec`. `num` is automatically promoted to `dec` by the codegen's `_coerce()` method using the `sitofp` instruction.

Operations: same as `num` but uses floating-point IR instructions (`fadd`, `fsub`, `fmul`, `fdiv`, `fcmp_ordered`).

`text` — null-terminated string

A read-only pointer to a null-terminated sequence of bytes. String literals are stored as global constants in the compiled binary.

```
var name : text := `svchost.exe`
var msg  : text := proc_name(procs, 0)
```

LLVM type: `i8*`. C equivalent: `const char*`.

String comparison uses `strcmp`, not pointer comparison. Two strings with the same content but stored at different locations would compare as not-equal with `==` on pointers. JOCKY's `is` operator on `text` values calls `strcmp` and checks if the result is 0.

Strings are immutable at the IR level. You cannot modify the characters of a string — you can only re-assign the variable to point to a different string.

`flag` — boolean

Represents `yes` (true) or `no` (false).

```
var done   : flag := no
var found  : flag := yes
var active : flag := x gt 0
```

LLVM type: `i1` (1-bit integer). 0 = false, 1 = true.

Operations: `also` (AND), `or` (OR), `flip` (NOT), `is`, `isnt`.

All comparison operators produce `flag`. `check` and `loop` conditions must be `flag`.

`raw` — opaque byte pointer

Used for values returned by the stdlib (process lists, network connections, buffers). You cannot perform arithmetic on `raw` directly — it is passed to other stdlib functions.

```
var procs : raw := procs_list()
var n     : num := proc_count(procs)
var name  : text := proc_name(procs, 0)
```

LLVM type: `i8*`. C equivalent: `void*`.

`nothing` — void

Only valid as a function return type. Functions that don't return a value are declared `-> nothing`. Cannot be used as a variable type.

```
func scan() -> nothing {
    report(`scanning`)
}
```

LLVM type: `void`. C equivalent: `void`.

Variable Declaration

Syntax: `var name : type := expression`

`:=` is the declaration assignment. It declares the variable AND sets its initial value in one statement. You cannot declare a variable without initialising it.

```
var x      : num  := 42
var pi     : dec  := 3.14159
var label  : text := `hello`
var active : flag := yes
var procs  : raw  := procs_list()
```

Scoping: Variables declared inside a `{ }` block are local to that block. They are not visible outside it. Variables declared in an outer block ARE visible in inner blocks.

```
var x : num := 1          ## visible everywhere in this function
check (yes) {
    var y : num := 2      ## only visible inside this block
    x <- x + y            ## OK — x is in outer scope
}
## y is NOT accessible here
```

Shadowing is not allowed. Redeclaring a name already declared in the exact same scope is a semantic error. Redeclaring in an inner scope would shadow the outer — this is also not supported.

Variable Update

Syntax: `name <- expression`

`<-` updates an existing variable. The variable must have been declared with `var` first.

```
var count : num := 0
count <- count + 1
count <- count * 2
```

Type must remain the same. You cannot change the type of a variable after declaration. `num` ↔ `dec` coercion is allowed (the semantic analyser permits it; the codegen converts automatically).

Functions

Definition

```
func name(param1 : type1, param2 : type2) -> return_type {
    body
}
```

Parameters are declared with `name : type`. Multiple parameters are comma-separated.

```
func add(a : num, b : num) -> num {
    give a + b
}

func greet(name : text) -> nothing {
    report(name)
}

func no_args() -> flag {
    give yes
}
```

Return — `give`

```
give expression    ## return with a value
give               ## return void (in a -> nothing function)
```

`give` exits the function immediately. Any code after `give` in the same block is unreachable and will be skipped. The compiler adds an implicit `give` at the end of every `-> nothing` function (returns 0/null/false at the end of non-void functions that fall off the end).

Calling functions

```
result <- add(3, 5)        ## call with args, capture return value
report(`hello`)            ## call void function as statement
var n : num := double(21)  ## call in expression context
```

Function calls can appear anywhere an expression is expected, or as standalone statements (for void functions). Arguments can be any expression — literals, variables, other function calls.

Mutual calls

Any function can call any other function in the same file, regardless of order. The compiler registers all function signatures in a first pass before generating any function body.

```
func is_even(n : num) -> flag {
    check (n is 0) { give yes }
    give is_odd(n - 1)        ## is_odd defined below — this is fine
}

func is_odd(n : num) -> flag {
    check (n is 0) { give no }
    give is_even(n - 1)
}
```

Conditionals

```
check (condition) {
    ## runs if condition is yes
}
```

```
check (condition) {
    ## runs if yes
} otherwise {
    ## runs if no
}
```

`condition` must be a `flag` expression (or any comparison). The `otherwise` clause is optional.

Nested conditionals:

```
check (x gt 10) {
    check (x gt 20) {
        report(`very big`)
    } otherwise {
        report(`big`)
    }
} otherwise {
    report(`small`)
}
```

Loops

```
loop (condition) {
    ## runs while condition is yes
}
```

While-loop semantics: The condition is checked BEFORE each iteration. If it is `no` on the first check, the body never runs.

Infinite loop:

```
loop (yes) {
    ## runs forever (or until stop)
}
```

`stop` — break

Exits the innermost loop immediately. Execution continues after the closing `}` of the loop.

```
var i : num := 0
loop (yes) {
    check (i is 10) {
        stop    ## exit when i reaches 10
    }
    i <- i + 1
}
```

`skip` — continue

Jumps to the next iteration of the innermost loop. The loop condition is re-checked.

```
var i : num := 0
loop (i lt 10) {
    i <- i + 1
    check (i mod 2 is 0) {
        skip    ## skip even numbers
    }
    report(`odd`)
}
```

Operators

Operator Precedence Table (highest to lowest)

Level	Operators	Associativity
6 (highest)	`flip`, unary `-`	Right
5	`*`, `/`, `mod`	Left
4	`+`, `-`	Left
3	`is`, `isnt`, `gt`, `lt`, `gte`, `lte`	None (not chainable)
2	`also`	Left
1 (lowest)	`or`	Left

Examples:

```
flip a also b             ## (flip a) also b
a + b * c                 ## a + (b * c)
x gt 0 also y lt 10      ## (x gt 0) also (y lt 10)
a or b also c             ## a or (b also c)   — 'also' binds tighter than 'or'
```

Arithmetic operators

Operator	Meaning	Types
`+`	Addition	num+num→num, dec+dec→dec, mixed→dec
`-`	Subtraction	same
`*`	Multiplication	same
`/`	Division	num/num→num (integer division), dec/dec→dec
`mod`	Remainder	num mod num → num
`-x`	Negation	-num→num, -dec→dec

Integer division truncates toward zero: `7 / 2` is `3`, not `3.5`. For decimal division, use `dec` type variables.

Comparison operators

All comparisons produce `flag`.

Operator	Meaning
`is`	Equal. For `text`: uses `strcmp`. For numbers: exact equality.
`isnt`	Not equal.
`gt`	Greater than (numeric only).
`lt`	Less than (numeric only).
`gte`	Greater than or equal (numeric only).
`lte`	Less than or equal (numeric only).

Comparisons are not chainable: `a gt b gt c` is a syntax error.

Boolean operators

Operator	Meaning	Short-circuit
`also`	Logical AND	No (both sides evaluated)
`or`	Logical OR	No (both sides evaluated)
`flip`	Logical NOT	—

JOCKY does not implement short-circuit evaluation. Both sides of `also` and `or` are always evaluated. This is a current limitation of the implementation.

Boolean Literals

```
yes    ## true  — IR: i1 with value 1
no     ## false — IR: i1 with value 0
```

The `empty` Literal

```
var p : raw := empty
```

`empty` represents a null pointer. It is useful for initialising a `raw` variable before assigning it a real value from a stdlib function. In IR, `empty` becomes `null` for pointer types.

Complete Example — Process Scanner

```
## forensic_scan.jk
## Full process scanning demonstration

func find_proc(target : text) -> flag {
    var procs : raw  := procs_list()
    var n     : num  := proc_count(procs)
    var i     : num  := 0
    var found : flag := no

    loop (i lt n) {
        var name : text := proc_name(procs, i)
        check (name is target) {
            found <- yes
            stop
        }
        i <- i + 1
    }
    give found
}

func count_procs() -> num {
    var procs : raw := procs_list()
    give proc_count(procs)
}

func kill_if_found(target : text) -> nothing {
    var procs : raw := procs_list()
    var n     : num := proc_count(procs)
    var i     : num := 0
    loop (i lt n) {
        var name : text := proc_name(procs, i)
        check (name is target) {
            var pid : num := proc_pid(procs, i)
            proc_kill(pid)
            stop
        }
        i <- i + 1
    }
}

func start() -> nothing {
    report(`JOCKY Forensic Tool`)
    report(`Scanning processes...`)

    var total : num := count_procs()

    check (find_proc(`svchost.exe`)) {
        report(`svchost.exe is running`)
    } otherwise {
        report(`svchost.exe not found`)
```

```
    }

    check (find_proc(`malware.exe`)) {
        report(`ALERT: malware.exe found - terminating`)
        kill_if_found(`malware.exe`)
    } otherwise {
        report(`System clean: malware.exe not present`)
    }

    report(`Scan complete`)
}
```

Common Mistakes

Mistake	Error	Fix
`var x : num = 5`	Parse error: expected `:=`	Use `:=` for declarations
`x = x + 1`	Parse error: unexpected `=`	Use `<-` for updates
`report("hello")`	Lexer error: unexpected `"`	Use backticks: `` `hello` ``
`check x gt 5 { }`	Parse error: expected `(`	Wrap condition: `check (x gt 5)`
`func f() { }`	Parse error: expected `->`	Add return type: `func f() -> nothing { }`
`var x := 5`	Parse error: expected `:`	Declare type: `var x : num := 5`
`give` in `-> num` function with no value	Semantic error	`give 0`
Using a variable before declaring it	Semantic error	Move `var` before first use
Calling `report(42)`	Semantic error: type mismatch	`report` requires `text`